

Agent Robot Q-Learning dans un environnement / UE Robotique et IA

Marius Casamian, cm22205773, Étudiant L3 Intelligence Artificielle unice

2 mai 2025

Résumé

Ce rapport présente l'implémentation d'un agent (robot) naviguant à travers un environnement (un labyrinthe) à l'aide de l'algorithme de Q-learning. Nous détaillons la construction du labyrinthe, la définition de l'environnement, le schéma d'apprentissage ect, ainsi que l'analyse des résultats grâce à des visualisations et des interprétations.

1 Prérequis et installation

Le projet nécessite Python 3 et les bibliothèques NumPy et Matplotlib. J'ai fais le choix de ne pas utiliser un notebook pour avoir plus de flexibilité dans le code et une arborescence plus claire et structurée. Aucun autre module externe n'est requis. Un interpréteur standard suffit pour exécuter les scripts. `python3 main.py` ou `python3 inspectQtable.py`

2 Arborescence du projet

L'organisation des fichiers est la suivante :

```
.
maze_layout.py          # construction du labyrinthe
main.py                 # entraînement et simulation Q-learning
compare_policy.py       # visualisation de la convergence de la politique
inspectQtable.py        # analyse des valeurs Q par action
visualisationQtable.py  # champ de vecteurs de la politique
q_table.npy             # table Q sauvegardée
outputs_images/         # dossier de sorties graphiques
heatmap_Q_*.png
value_map_Vs.png
policy_vector_field_enhanced.png
compare_policy.png
```

3 Explication des scripts

main.py contient l'agent d'apprentissage principal : entraînement, simulation et affichage du parcours.

maze_layout.py définit la configuration statique du labyrinthe (murs et position de la base).

compare_policy.py observe l'évolution temporelle de la politique en sauvegardant des instantanés de la table Q .

inspectQtable.py génère les heatmaps des valeurs $Q(s, a)$ pour chaque action.

visualisationQtable.py représente la politique gloutonne finale sous forme de champ de vecteurs.

q_table.npy contient la table Q sauvegardée après apprentissage.

outputs_images/ regroupe toutes les figures générées automatiquement pour analyse.

4 Description du labyrinthe

Le labyrinthe est une grille de dimension 20×22 , dont les murs ont été codés « à la main » pour reproduire fidèlement la configuration suivante (j'ai essayé sur la carte de l'énoncé, mais j'ai voulu varier pour essayer d'autres configurations) :

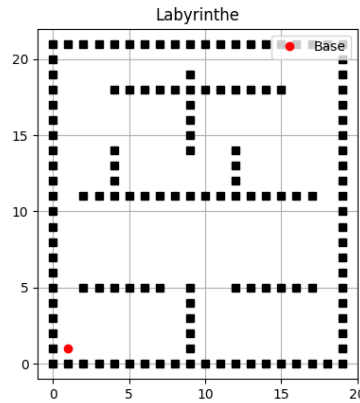


FIGURE 1 – Plan du labyrinthe (mur extérieur, barrières internes).

La position d'arrivée de l'agent, appelée « base », est fixée en $(1,1)$. Les murs sont codés comme un ensemble de coordonnées (x, y) interdites et le spawn du robot est aléatoire.

5 Formalisation de l'environnement

À chaque état s (coordonnée libre) l'agent peut exécuter quatre actions $a \in \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$. Les transitions sont déterministes : une tentative de franchir un mur laisse l'agent sur place et lui inflige une pénalité de -5 . Tout déplacement valide coûte -1 , et atteindre la base rapporte $+100$.

6 Algorithme Q-learning

La table $Q(s, a)$ est initialisée aléatoirement dans $[-1, 1]$. À chaque épisode, l'agent part d'un état aléatoire non mural, puis choisit ses actions selon une stratégie ε -greedy dont ε décroît de $0,3$ à $0,01$ sur 2000 épisodes. La mise à jour s'effectue selon :

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$$

avec $\alpha = 0,2$ et $\gamma = 0,95$. Nous recueillons pour chaque épisode la récompense cumulée, le nombre de collisions et la réussite (atteinte de la base).

Les hyperparamètres choisis ($\alpha = 0.2$, $\gamma = 0.95$, décroissance exponentielle de ε) influencent la rapidité et la stabilité de l'apprentissage. Des tests ont montré qu'un taux d'apprentissage trop élevé déstabilisait les valeurs Q , et qu'un γ inférieur ralentissait la propagation des récompenses.

```
Ép 1/2000 | R=-136.0 | steps=100 | colls=9
Ép 50/2000 | R=18.0 | steps=75 | colls=2
Ép 100/2000 | R=95.0 | steps=6 | colls=0
Ép 150/2000 | R=-116.0 | steps=100 | colls=4
Ép 200/2000 | R=73.0 | steps=20 | colls=2
Ép 250/2000 | R=-108.0 | steps=100 | colls=2
Ép 300/2000 | R=-128.0 | steps=100 | colls=7
Ép 350/2000 | R=-108.0 | steps=100 | colls=2
Ép 400/2000 | R=-18.0 | steps=99 | colls=5
Ép 450/2000 | R=92.0 | steps=9 | colls=0
Ép 500/2000 | R=-124.0 | steps=100 | colls=6
Ép 550/2000 | R=-31.0 | steps=100 | colls=8
Ép 600/2000 | R=-108.0 | steps=100 | colls=2
Ép 650/2000 | R=61.0 | steps=36 | colls=1
Ép 700/2000 | R=55.0 | steps=42 | colls=1
Ép 750/2000 | R=72.0 | steps=21 | colls=2
Ép 800/2000 | R=95.0 | steps=6 | colls=0
Ép 850/2000 | R=66.0 | steps=27 | colls=2
Ép 900/2000 | R=-100.0 | steps=100 | colls=0
Ép 950/2000 | R=77.0 | steps=24 | colls=0
Ép 1000/2000 | R=94.0 | steps=7 | colls=0
Ép 1050/2000 | R=89.0 | steps=12 | colls=0
Ép 1100/2000 | R=-116.0 | steps=100 | colls=
Ép 1150/2000 | R=79.0 | steps=22 | colls=0
Ép 1200/2000 | R=73.0 | steps=24 | colls=1
Ép 1250/2000 | R=78.0 | steps=23 | colls=0
Ép 1300/2000 | R=72.0 | steps=29 | colls=0
Ép 1350/2000 | R=51.0 | steps=50 | colls=0
Ép 1400/2000 | R=77.0 | steps=24 | colls=0
Ép 1450/2000 | R=86.0 | steps=11 | colls=1
Ép 1500/2000 | R=97.0 | steps=4 | colls=0
Ép 1550/2000 | R=100.0 | steps=1 | colls=0
Ép 1600/2000 | R=55.0 | steps=34 | colls=3
Ép 1650/2000 | R=70.0 | steps=31 | colls=0
Ép 1700/2000 | R=88.0 | steps=13 | colls=0
Ép 1750/2000 | R=100.0 | steps=1 | colls=0
Ép 1800/2000 | R=69.0 | steps=32 | colls=0
Ép 1850/2000 | R=70.0 | steps=31 | colls=0
Ép 1900/2000 | R=77.0 | steps=24 | colls=0
```

FIGURE 2 – Exemple de logs à l'exécution du main.py

On peut voir que le nombre de collisions diminuent à mesure de l'apprentissage. Vu que c'est aléatoire dans l'apparition du robot certains ne font que très peu de "steps", mais c'est fonctionnel. Le but est que le robot apprenne par essais erreurs donc il est normal qu'il percute des murs au début, par contre à vérifier qu'il se comporte bien vers la fin des itérations.

7 Visualisation de l'apprentissage

Après l'entraînement, nous extrayons la politique gloutonne $\pi(s) = \arg \max_a Q(s, a)$. Les figures suivantes illustrent l'évolution de Q et de la valeur d'état $V(s) = \max_a Q(s, a)$.

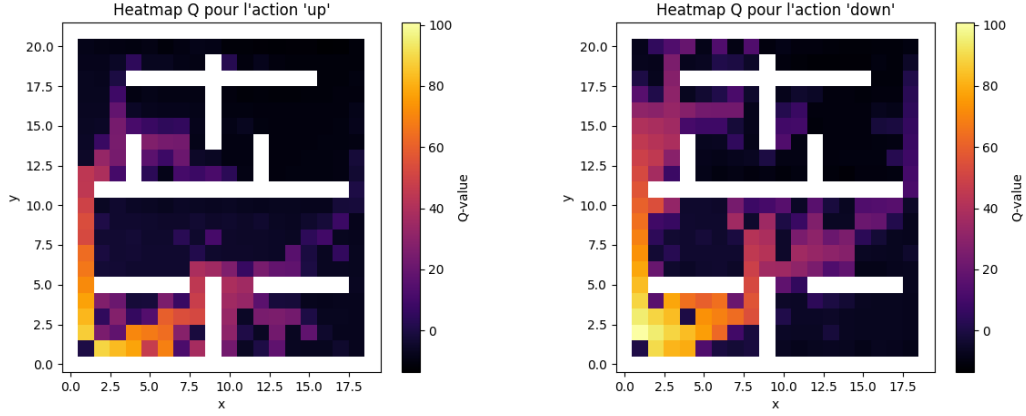


FIGURE 3 – Heatmaps des valeurs $Q(s, a)$ pour les actions « haut » et « bas ».

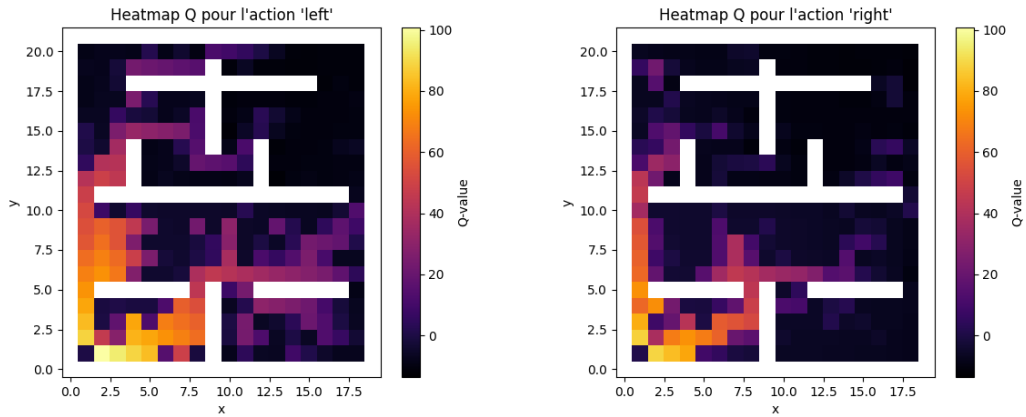


FIGURE 4 – Heatmaps des valeurs $Q(s, a)$ pour les actions « gauche » et « droite ».

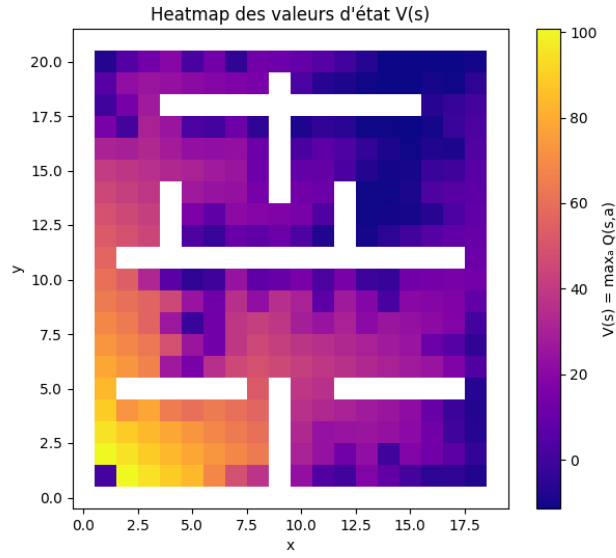


FIGURE 5 – Heatmap de $V(s) = \max_a Q(s, a)$.

Le champ vectoriel de la politique (couleur selon Q , épaisseur selon la proximité de la base) met en évidence les directions préférentielles à chaque position :

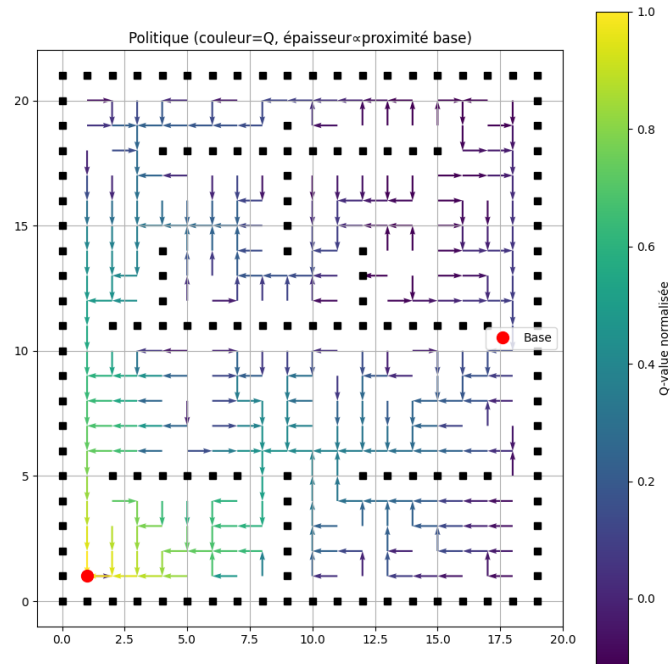


FIGURE 6 – Champ de vecteurs de la politique gloutonne après entraînement.

8 Comparaison de la politique

Pour observer la convergence, nous capturons la politique aux épisodes 1, 100, 500, 1000 et 2000. Les sous-figures ci-dessous illustrent la montée en compétence de l'agent :

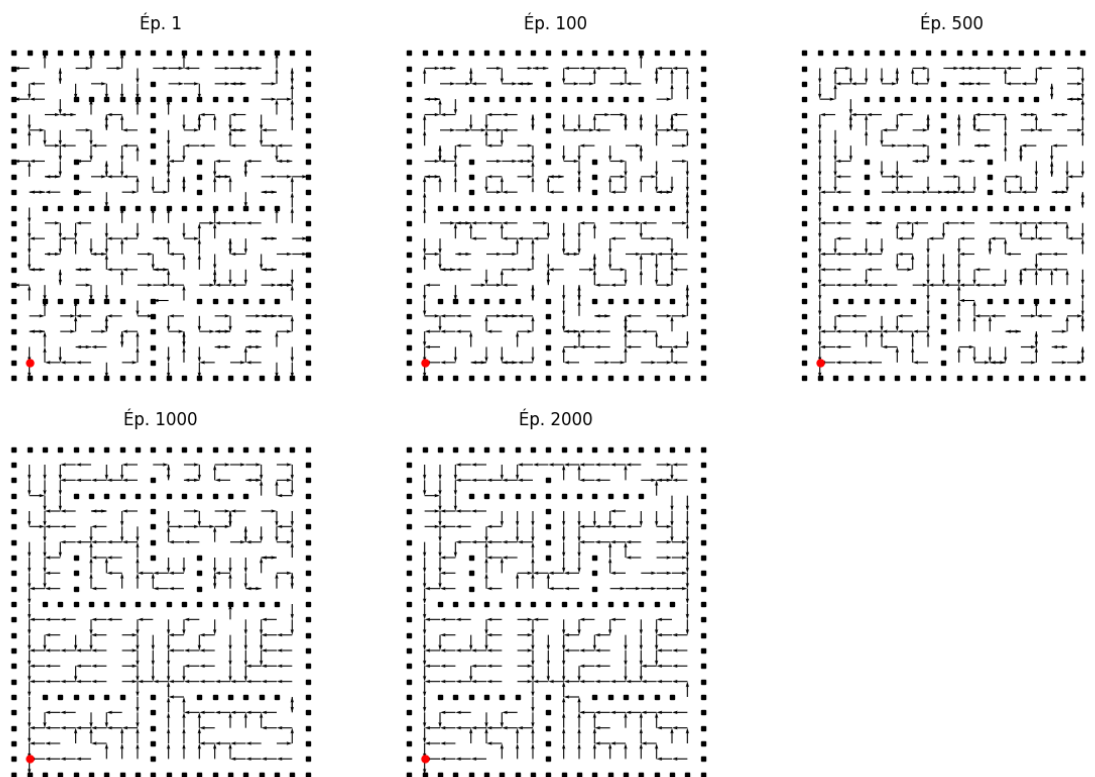


FIGURE 7 – Évolution de la politique gloutonne à différentes étapes de l'apprentissage.

Des comportements incohérents au début puis vers 1000 une politique plus intéressante.

9 Évolution du taux de réussite

Au lieu de nous focaliser sur la récompense brute, nous mesurons désormais la « réussite » de l'agent par le nombre d'épisodes où il parvient à atteindre la base. La figure 8 présente deux courbes : – le taux de réussite cumulé (en orange) qui correspond simplement à la fraction d'épisodes gagnants jusqu'à l'instant t , – la moyenne glissante (en bleu), calculée sur une fenêtre de 50 épisodes, qui lisse les variations locales.

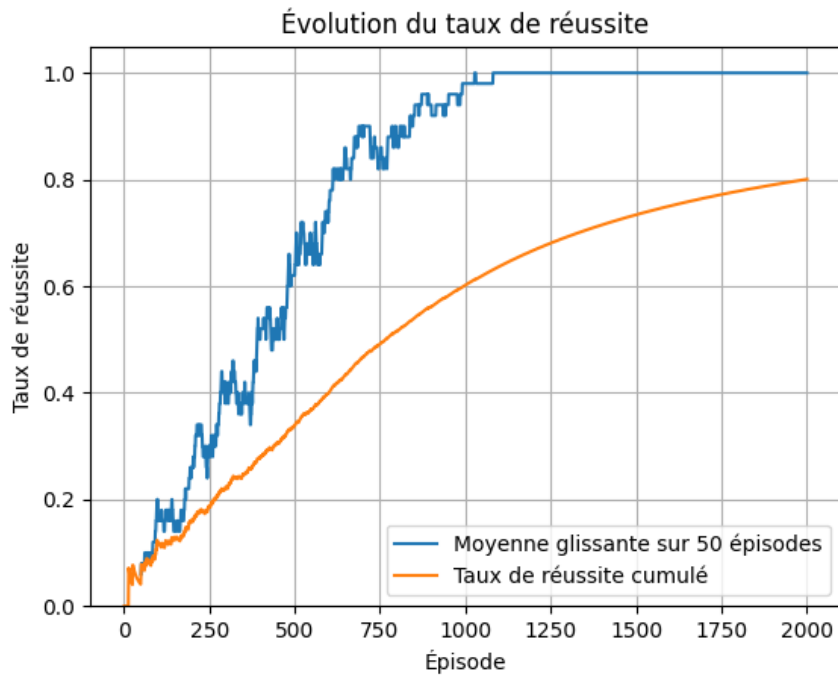


FIGURE 8 – Évolution du taux de réussite : courbe cumulée et moyenne glissante sur 50 épisodes.

On observe que le robot démarre avec un taux très faible (presque 0%), puis progresse rapidement pour dépasser les 80 % vers l'épisode 700. La moyenne glissante atteint 100 % peu avant l'épisode 1200, signe d'une politique stable et fiable. Cette métrique est plus robuste que la simple somme des récompenses, car elle ne dépend pas de la distance de départ au moment du spawn.

10 Interprétation des résultats

L'agent parvient rapidement à éviter les murs et à se diriger vers la base. Les heatmaps montrent localement les régions où certaines actions sont plus valorisées. La carte de valeurs d'état révèle que la région proche de la base est celle qui porte les valeurs maximales, conformément à la récompense positive, et enfin le taux de réussite vient confirmer la robustesse du robot après un certain nombre d'époques.

11 Conclusion

Ce projet illustre efficacement l'application du Q-learning à un problème de navigation. La structuration modulaire du code permet de distinguer clairement la définition du labyrinthe, l'apprentissage et la visualisation. Les résultats sont cohérents avec les attentes théoriques et ouvrent la voie à des extensions vers des environnements plus complexes ou à l'intégration de fonctions de récompense plus sophistiquées. Ce projet m'a permis en parallèle d'approfondir pour un autre projet de multi-agents dans la modélisation du trafic Urbain et plus précisément pour des jeux MDP (valueIteration et Qlearning).

Lien GitHub du projet

Le code source complet, les scripts de visualisation et les figures sont disponibles ici :

12 Limites et perspectives

Le modèle actuel reste simple : un seul agent, environnement statique, récompenses fixes. On pourrait enrichir l'expérience avec : des obstacles mobiles, des pénalités variables, ou du Q-learning distribué avec plusieurs agents concurrents.